Pengantar Web dengan Golang

Pemrograman Web • 13 slides • Generated on 11 October 2026, 22:31

Persiapan lingkungan kerja, membuat hello world.

Pengantar Web dengan Go

- **Kebutuhan untuk pembelajaran:**
  - Laptop (spec standar, RAM 8GB cukup)
  - Go terbaru, download di https://go.dev/dl/
  - VS Code + extension "Go" by Google
  - Terminal/command prompt yang berfungsi

Peraturan Kelas

Slide 2

- **Mahasiswa diharapkan:**
  - mengikuti kuliah dengan tertib
  - mempraktekkan apa yang disampaikan
  - menyampaikan/bertanya bila ada kesulitan
- **Penilaian:**
  - Bobot penilaian ada di dalam RPS
  - UTS dan UAS essay, mengerjakan di kelas
  - Nilai project bobot 30%, tapi tidak berarti project bagus = A

Apa itu Web Programming?

Slide 3

- Definisi singkat:
  - **Web programming = buat program yang bisa diakses lewat browser/HTTP**
  - Bukan hanya "buat web" — tapi **membuat servis yang merespons request**
- Waktu kamu buka tokopedia.com, yang terjadi:
  - Browser mengirim **request** ke server Tokopedia
  - Server Tokopedia memproses: "ini mau lihat apa?"
  - Server kirim kembali **response** (HTML/JSON/gambar)
  - Browser menampilkan (render) itu jadi halaman yang terlihat
- Dalam satu aplikasi web modern, biasanya ada 3 layer:
  - **Frontend** — yang kita lihat di browser (HTML, CSS, JS)

    - **Backend** — yang proses data & bisnis (Go, PHP, Python, JS, Rust, dsb.)
    - **Database** — tempat simpan data (MySQL, PostgreSQL, MongoDB, Redis)

- Di mata kuliah ini, **kita fokus di backend pakai Go**
    - Frontend-nya pakai HTML sederhana, server side rendering (SSR)
    - Database pakai MySQL, nanti di pertemuan 6+
    - Goal: paham **server-side web** menggunakan Golang

Anatomi Web: Browser, Server, dan Segala yang di Tengahnya

Slide 4

- Setiap kali kita akses website, ada **minimal 3 pihak** yang bermain:
  - **Client** — yang minta sesuatu (browser, app mobile, curl, Postman)
  - **Server** — yang memberikan sesuatu (program go/PHP/Rust/dsb)
  - **Jaringan** — jalan tol penghubung (internet/WiFi/LAN)
- **Cara browser berkomunikasi dengan server** lewat protokol **HTTP**
  - HTTP = HyperText Transfer Protocol
  - Bisa dibayangkan seperti "bahasa komunikasi" antara client & server
  - HTTPS = HTTP tapi di-encrypt (ada SSL/TLS)
- Hal yang kita lakukan sebagai web developer:
  - **bagaimana server memproses request** (di sinilah Go bekerja)

    - **apa yang dikembalikan ke client** (HTML, JSON, file, dll)

- Coba inspect salah satu website:
    - Buka tokopedia.com
    - Klik kanan -> Inspect -> Network
    - Refresh, lihat request yang jalan

HTTP

Slide 5

- HTTP punya **method** — verb untuk memberitahu server apa yang diminta:
  - `GET` — meminta data (default, paling sering digunakan)
  - `POST` — mengirim data baru (form, pendaftaran akun)
  - `PUT` / `PATCH` — memperbarui data yang sudah ada
  - `DELETE` — menghapus data
- HTTP punya **status code** — angka yang menginformasikan hasil request:
  - **2xx** = berhasil (200 OK, 201 Created)
  - **3xx** = redirect (301 Moved Permanently)
  - **4xx** = kesalahan client (404 Not Found, 401 Unauthorized, 403 Forbidden)
  - **5xx** = kesalahan server (500 Internal Server Error, 503 Service Unavailable)
- **URL anatomy** — struktur yang sering digunakan:

```
https://api.example.com:443/v1/users/42?active=true#section-2
?       ?               ?   ?  ?     ?  ?               ?
?       ?               ?   ?  ?     ?  ?               ?? Fragment (hanya di browser, tidak dikirim ke server)
?       ?               ?   ?  ?     ?  ?? Query string
?       ?               ?   ?  ?     ?? Resource ID
?       ?               ?   ?  ?? Collection name (REST style)
?       ?               ?   ?? API version
?       ?               ?? Port (443 default untuk HTTPS)
?       ?? Host
?? Scheme/protocol
```

- **Header** — metadata tambahan pada request/response:
  - `Content-Type: application/json` — menunjukkan format data
  - `Authorization: Bearer abc123` — mengirim token login
  - `User-Agent: Chrome/120` — identitas browser

Mengapa Go?

Slide 6

- Go menawarkan beberapa keunggulan untuk pengembangan web backend:
  - **Cepat**: dikompilasi menjadi single binary, berjalan langsung di OS
    - PHP memerlukan interpreter (Apache/Nginx + PHP-FPM) di tengah
    - Perbandingan kasar: Go dapat menangani 10–100x lebih banyak request/detik daripada PHP
  - **Concurrency first-class**: `go func()` sudah cukup untuk eksekusi konkuren
  - **Static binary**: deploy cukup salin 1 file, langsung jalan
  - **Standard library `net/http` cukup powerful**: tidak butuh framework untuk web server
- Adopsi Go di industri Indonesia:
  - Tokopedia, Gojek, Bukalapak, Traveloka — menggunakan Go di production
  - Lowongan pekerjaan Go di Indonesia terus meningkat

- Pertimbangan trade-off:
  - Ekosistem web Go lebih kecil dibanding PHP/Node.js
  - Untuk CMS, PHP/Laravel masih menjadi pilihan utama
  - Untuk **API, microservice, CLI tool, sistem dengan kebutuhan performa** — Go lebih unggul

Seperti apa kode Golang?

Slide 7

- Perbandingan dengan PHP:
  - **PHP**:
    - kode PHP diletakkan di folder web server, PHP yang menjalankan kode
    - setiap kali ada request, kode akan dijalankan ulang dari awal sampai menghasilkan response
  - **Go**:
    - program Go berjalan sendiri sebagai web server
    - Tidak memerlukan Apache, Nginx, PHP-FPM, atau komponen lain
- Gambaran awal kode Golang:

```
// main.go
package main
```

```
import "net/http"

func main() {
    // Daftarkan handler untuk path "/"
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Hello, Web!")
    })

    // Siap serve di port 8080"
    http.ListenAndServe(":8080", nil)
}
```

- Yang terjadi saat `go run main.go`:

    1. Kode dikompilasi menjadi binary
    2. Binary berjalan
    3. Di dalamnya terdapat **HTTP server** yang listen di port 8080
    4. Siap menerima request dari browser
    5. Dihentikan dengan Ctrl+C

- Deployment pattern di Go:

- Program Go dapat sekaligus berfungsi sebagai:
  - Web server
  - CLI tool
  - Background worker
- Deployment cukup menyalin 1 file binary, lalu jalankan

Instalasi Go

Slide 8

- Unduh installer dari https://go.dev/dl/
  - Pilih sesuai OS: `.msi` (Windows), `.pkg` (macOS), tarball (Linux)
  - Ambil versi **stable terbaru**
- Verifikasi instalasi (buka terminal baru):

```
go version
```

- Output yang diharapkan: `go version go1.22.x ...`
- Jika "command not found", coba restart terminal

Setup Editor: VS Code + Go Extension

Slide 9

- Langkah instalasi extension:

  1. Buka VS Code
  2. Klik icon **Extensions** di sidebar (atau `Ctrl+Shift+X`)
  3. Cari **"Go"** di kotak pencarian
  4. Pilih **"Go" by Go Team at Google** (bukan "Go Nightly")
  5. Klik **Install**

- Konfigurasi yang disarankan (`settings.json`):

```
{
  "go.useLanguageServer": true,
  "go.formatTool": "goimports",
  "editor.formatOnSave": true
}
```

- `goimports` menambahkan import secara otomatis
- Format on save → kode selalu rapi, tidak perlu pusing dengan styling
- Verifikasi instalasi:
  - Buat folder kosong, buat file `main.go` dengan `package main`
  - Akan muncul popup "Install Go tools?" → klik Yes
  - Tunggu hingga selesai (1–2 menit)
  - Autocomplete, go-to-definition, error highlighting sudah aktif

Struktur Folder Project Go

Slide 10

- Go **tidak memaksakan** struktur folder tertentu
  - Terdapat konvensi yang baik untuk project serius
  - Untuk saat ini, kita mulai dari struktur paling sederhana
- Struktur minimal project Hello Web:

```
hello-web/
??? go.mod          <- "identitas" project
??? go.sum          <- lock file untuk dependency (jika ada)
??? main.go         <- entry point program
```

- Membuat project baru:

```
mkdir hello-web
cd hello-web
go mod init github.com/username/hello-web
```

- Kapan `go.sum` dibuat?
  - Saat menggunakan library eksternal
  - Ketika belum ada dependency, `go.sum` belum dibuat
  - Akan dibuat otomatis pada unduhan dependency pertama
- Contoh struktur project Go:

```
web-app/
??? go.mod
??? go.sum
??? main.go                 <- entry point
??? internal/               <- kode hanya untuk project ini
?   ??? handler/
?   ??? service/
?   ??? repository/
??? pkg/                    <- kode yang dapat di-import project lain (di luar internal)
    ??? utils/
```

- `internal/` — Go compiler otomatis mencegah impor dari luar project
- `pkg/` — kode yang dapat di-share (nama folder tidak harus "pkg")

Perintah Dasar: `go run` vs `go build`

Slide 11

- Dua perintah yang akan digunakan 90% waktu:
- `go run main.go` — kompilasi sementara, langsung jalan, hapus binary setelah selesai

```
$ go run main.go
Starting server on :8080
# setelah Ctrl+C, binary dihapus
```

- Cocok untuk **development** — melihat hasil dengan cepat
- Tiap kali dijalankan, compiler akan memproses perubahan -> menjadi binary baru
- `go build` — kompilasi menjadi binary, jalankan manual

```
$ go build -o hello-web
$ ls
hello-web  main.go  go.mod
$ ./hello-web
```

```
Starting server on :8080
```

- Cocok untuk **production** — deploy ke server, distribusikan
- File `hello-web` adalah binary yang dapat disalin ke server lain
- Target OS mengikuti OS saat kompilasi (atau bisa cross-compile untuk platform lain)

```
GOOS=linux GOARCH=amd64 go build -o hello-web
```

- Variasi `go run`:

```
go run main.go              # jalankan 1 file
go run .                    # jalankan semua file .go di folder
go run *.go
```

Praktek: Hello Web

- Buat file `main.go` di folder `hello-web/`, isi dengan kode berikut:

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
 // Pasang handler ke URL path
 http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
  // Agar di terminal terlihat ada yang mengakses
  fmt.Println("Ada yang akses:", r.Method, r.URL.Path)

  // Tulis response ke browser
  fmt.Fprint(w, "Halo! Anda berhasil akses web server Go!")
 })

 // Informasikan server sedang berjalan
 fmt.Println("Server berjalan di http://localhost:8080")
 fmt.Println("Tekan Ctrl+C untuk menghentikan")
```

```
 // Mulai web server di port 8080
 http.ListenAndServe(":8080", nil)
}
```

- Beberapa aturan penulisan Go:
  - Tidak ada `;` di akhir baris
  - Opening brace `{` harus di akhir baris yang sama

Menjalankan Server & Menguji di Browser

- Langkah 1: kompilasi dan jalankan

```
$ go run main.go
Server berjalan di http://localhost:8080
Tekan Ctrl+C untuk menghentikan
```

- Server telah berjalan, **jangan tutup terminal**
- Langkah 2: uji di browser
  - Buka browser, akses `http://localhost:8080`
- Langkah 3: hentikan server
  - Kembali ke terminal yang menjalankan server
  - Tekan `Ctrl+C`